How the project uses D-Wave

Code walk-through, modes of use, checks, estimates, budget guard, and the 1-second test

`Projects_master_HPQC/scripts/QuantumComputing/Dimod`

Abstract

This document explains, file by file and line by line, how the SK-game campaign talks to D-Wave:

- which Ocean functions it uses, what they can do, and how the code harnesses them;
- the six ways ("modes") of running an annealer in this project;
- what `qpu --check` actually does, and how the QPU time is estimated (three levels, with the formulas);
- how the budget guard (`--budget-seconds`) makes sure a run never exceeds a given QPU time;
- how the results are turned into success probabilities, time-to-solution and the Zephyr/Pegasus comparison;
- the package `dwave_utils`: the linear pipeline (budget, calibrate, run, report, compare), the feasibility model, and the fit of the machine's control error σ and effective temperature β_{eff}.

The code excerpts are read directly from the source files when the document is compiled, so the line numbers are those of the version in the repository. Section 17 gives the exact commands for a test that uses at most one second of QPU time, followed by a plan to spend the 180 s of this month before 4 November.

Contents

1 Where the D-Wave code lives **2**

2 The Ocean SDK: what each package can do, and what we use it for **2**
2.1 `dwave-cloud-client`: connection, solvers, configuration, timing model . . . 2
2.2 `dwave-system`: samplers for the QPU, composites, mock . . . 3
2.3 `dimod`: the problem and the results . . . 4
2.4 `minorminer` and `dwave-graphs`: embeddings and hardware graphs . . . 5
2.5 `dwave.embedding` (inside `dwave-system`): chains in and out . . . 5
2.6 `dwave-samplers` and `dwave-preprocessing` . . . 6

3 The modes of use **6**

4 Life of one QPU call, line by line **7**

5 What `qpu --check` does (and why it costs nothing) **10**

6 How QPU time is estimated: three levels **11**
6.1 Level 1: the rough plan (`main_simulation.py plan`) . . . 11
6.2 Level 2: the solver's timing model (`--check`, budget guard) . . . 12
6.3 Level 3: the charge (`qpu_access_time`) . . . 12

7 The budget guard: `qpu --budget-seconds S` **12**

8 Broken chains, read by read **14**

9 Ground states **16**

10 Chain strength and coupling scale: what is set, and what is not known **17**

11 Choosing the chain strength for every N: `coupling-scan` **18**

12 The QPU pipeline: `dwave_utils` **18**
12.1 Calibration: one p_{opt} per (machine, N), shared by every problem class 19
12.2 Spending the whole budget: `budget --fill` and `run --total` 19
12.3 Report and compare 19
12.4 Feasibility: can the QPU represent the problem at all? 20
12.5 Measuring σ and β_{eff}: `noise-fit` 21

13 Reproducibility: everything needed to reconstruct a call **22**

14 Ising or QUBO? And is the problem trivial? **24**

15 From reads to results **25**

16 How the D-Wave path was tested **26**

17 The one-second test, and a plan for the 180 s **26**
17.1 Test: at most 1 s of QPU time, both topologies 26
17.2 Spending the 180 s before 4 November 27

1 Where the D-Wave code lives

All paths are relative to `Dimod/src/unlearnable_games_explanation/`.

file	role
`skgame/optim/problem.py`	the SK problem W; `to_bqm()` builds the dimod model
`skgame/optim/embedding.py`	topologies (Zephyr, Pegasus), embedding strategies (clique, heuristic), chain statistics, embedding cache
`skgame/optim/backends.py`	`AnnealBackend`: one object per `device:topology:strategy`; builds the sampler, embeds, samples, collects statistics, estimates time
`skgame/optim/quantum.py`	`QuantumAnnealingSolver`: the older single-solver path (`DWaveCliqueSampler`/`EmbeddingComposite`), used by the planner benchmark
`skgame/optim/solvers.py`	`DimodSamplerSolver` (generic dimod wrapper) and the classical dimod samplers
`dwave_utils/pipeline.py`	the QPU pipeline: `budget`, `calibrate`, `run`, `report`, `compare` (Sec. 12)
`dwave_utils/coupling_scan.py`	the chain-strength scan and choice used by `calibrate`
`dwave_utils/feasibility.py`	yes / marginal / no: can the QPU represent a problem faithfully?
`dwave_utils/noise_fit.py`	σ and β_{eff} measured from the QPU's own small-N reads
`dwave_utils/shimming.py`	flux-bias shimming
`main_simulation.py`	the campaign: stages `init`, `instances`, `classical`, `embed`, `qpu`/`sim`, `analyze`; the QPU budget guard; the analysis
`tests/test_embedding.py`, `tests/test_campaign.py`	tests of all the above, including the QPU path against D-Wave's mock QPU

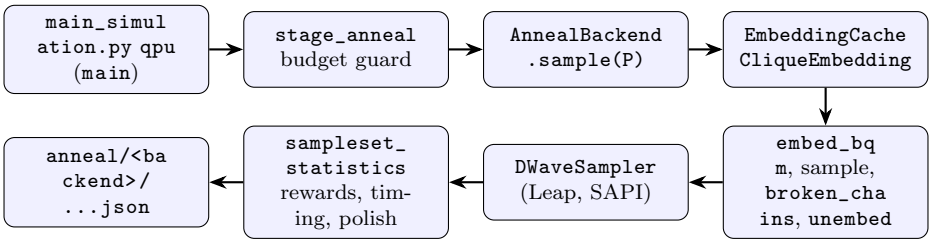


Figure 1: The path of one QPU call through the code.

2 The Ocean SDK: what each package can do, and what we use it for

D-Wave's Python SDK (*Ocean*) is a set of packages, all installed by `cineca/requirements.txt`. For each package: what it offers, then exactly how this project uses it.

2.1 `dwave-cloud-client`: connection, solvers, configuration, timing model

Capabilities.

- it talks to the Leap *Solver API* (SAPI) over HTTPS;
- it reads the token from the configuration file or from the environment (`DWAVE_API_TOKEN`);
- it selects solvers with *feature filters* such as `solver={"topology__type": "zephyr"}`;
- it exposes every solver's `properties`, including the timing data;
- it estimates the cost of a problem (`estimate_qpu_access_time`);
- it submits problems asynchronously;

- it provides the `dwave` command line: `config`, `auth`, `leap`, `solvers`, `ping`, `setup`.

How we use it.

- Through `DWaveSampler` (next subsection), which wraps a cloud-client `Solver`.
- `solver.name` identifies the QPU in the logs and keys the embedding cache, e.g. `Advantage2_system1`.
- `solver.estimate_qpu_access_time` is the second level of our time estimate (§6).
- On the command line: `dwave config create`, `dwave setup --oob`, `dwave solvers -la`, `dwave ping` (`Dimod/LEAP_TOKEN.md`).

2.2 `dwave-system`: samplers for the QPU, composites, mock

Capabilities.

- `DWaveSampler`: a dimod sampler bound to one structured QPU. It exposes `nodelist`/`edgelist` (the working qubits and couplers), `properties`, `parameters` (the accepted keyword arguments) and `to_networkx_graph()`. Its `sample()` accepts `num_reads`, `annealing_time`, `anneal_schedule`, `auto_scale`, `answer_mode`, `h_gain_schedule`, `anneal_offsets`, `programming_thermalization`, `readout_thermalization`, ...
- Composites that turn a structured sampler into one that accepts any problem:
  - `EmbeddingComposite` finds a heuristic embedding on every call;
  - `FixedEmbeddingComposite` uses a given embedding;
  - `LazyFixedEmbeddingComposite` finds one embedding and reuses it;
  - `DWaveCliqueSampler` uses cached all-to-all clique embeddings, scales with the extended J range and computes chain strengths.
- `dwave.system.testing.MockDWaveSampler`: an offline imitation of a QPU of any topology and size, which returns realistic timing fields.

How we use it.

- `DWaveSampler(solver={"topology__type": ...})` picks a Zephyr or Pegasus QPU (code below).
- `to_networkx_graph()` gives the *real* working graph. The embedding is computed on it, so it avoids dead qubits.
- Every QPU call does explicitly what `FixedEmbeddingComposite` does: `embed_bqm`, then `DWaveSampler.sample`, then `unembed_sampleset`, with our cached clique embedding. Doing it by hand keeps the *physical* reads, from which the broken chains of every read are counted (§8). The code was checked to give read-for-read the same results as the composite. The Zephyr and Pegasus back-ends go through identical code and differ only in hardware and embedding.
- `DWaveCliqueSampler` and `EmbeddingComposite` remain available in `QuantumAnnealingSolver` (§3).
- `MockDWaveSampler` is used in the tests and in the dry runs.

`skgame/optim/backends.py`, lines 88–118: building the child sampler (QPU, embedded simulator or logical simulator) and reading its hardware graph

```
88      # ------------------------------------------------------------- hardware graph ----
89      @property
90      def child(self):
91          """The structured sampler the embedding maps onto."""
92          if self._child is None:
93              if self.device == "qpu":
94                  from dwave.system import DWaveSampler
95                  s = self.settings
96                  self._child = DWaveSampler(solver=s.solver_name or self.topology.solver_filter)
97              elif self.embedded:
98                  import dimod
99                  from dwave.samplers import SimulatedAnnealingSampler
100                 G = self._target = self.topology.ideal_graph()
101                 self._child = dimod.StructureComposite(SimulatedAnnealingSampler(), list(G.nodes), list(G.edges)
    )
102             else:
103                 from dwave.samplers import SimulatedAnnealingSampler
104                 self._child = SimulatedAnnealingSampler()
105         return self._child
106
107     @property
108     def target(self):
109         """networkx graph of the working qubits and couplers."""
110         if self._target is None and self.embedded:
111             child = self.child                # building a sim child also sets the ideal graph
112             if self._target is not None:
113                 pass
114             elif hasattr(child, "to_networkx_graph"):
115                 self._target = child.to_networkx_graph()   # real working graph, keeps the topology metadata
116             else:
117                 raise ValueError(f"{self.spec}: cannot read the hardware graph of {type(child).__name__}")
118         return self._target
```

2.3 `dimod`: the problem and the results

Capabilities.

- `BinaryQuadraticModel` in SPIN or BINARY form;
- `SampleSet`, which holds the reads: `record.sample`, `record.energy`, `record.num_occurrences`, extra vectors such as `chain_break_fraction`, plus `info` (timing, embedding context). It also offers `aggregate()`, `first` and `resolve()`;
- a common `Sampler` interface (`sample(bqm, **kwargs)`, `parameters`);
- helper composites such as `StructureComposite`, and reference samplers such as `ExactSolver`.

How we use it.

- The SK problem becomes an Ising BQM whose energy is exactly $H(S) = -\sum_{i<j} W_{ij} S_i S_j$.
- `StructureComposite` restricts simulated annealing to the Zephyr or Pegasus graph (the `sim:*:clique` back-ends).
- `ExactSolver` checks the statistics code in the tests.

`skgame/optim/problem.py`, lines 45–50: the SK problem as a dimod BQM ($h = 0$, $J_{ij} = -W_{ij}$)

```
45      def to_bqm(self):
46          """dimod BinaryQuadraticModel (SPIN) with E = H(S). Lazy import."""
47          import dimod
48          iu = np.triu_indices(self.N, 1)
49          quad = {(int(i), int(j)): -float(self.W[i, j]) for i, j in zip(*iu)}
50          return dimod.BinaryQuadraticModel({}, quad, 0.0, dimod.SPIN)
```

2.4 `minorminer` and `dwave-graphs`: embeddings and hardware graphs

Capabilities.

- `minorminer.busclique.find_clique_embedding` returns all-to-all embeddings made of line segments, optimised for chain length and avoiding defects, with an on-disk cache;
- `busgraph_cache(G).largest_clique()` gives the largest K_N the graph supports;
- `minorminer.find_embedding` is the general heuristic;
- `dwave.graphs` (formerly `dwave_networkx`) provides `zephyr_graph`, `pegasus_graph`, coordinate converters and drawing functions.

How we use it.

- The clique strategy and its maximum size (below), which give the `largest_problem` printed by `qpu --check`.
- The heuristic baseline.
- The ideal graphs for the simulator and for the figures in `visualization/embedding_explanation`.

`skgame/optim/embedding.py`, lines 101–115: the all-to-all strategy: embedding and largest clique

```
101 class CliqueEmbedding(EmbeddingStrategy):
102     """Native all-to-all embedding (minorminer.busclique): every chain has the same, minimal-ish
103     length (Zephyr: about N/16 + 2 qubits; Pegasus: about N/12 + 2), deterministic and fast.
104     This is what DWaveCliqueSampler uses internally."""
105
106     name = "clique"
107
108     def embed(self, N, target):
109         from minorminer.busclique import find_clique_embedding
110         emb = find_clique_embedding(N, target, use_cache=True)
111         return {int(k): [int(q) for q in v] for k, v in emb.items()}
112
113     def max_size(self, target):
114         from minorminer.busclique import busgraph_cache
115         return len(busgraph_cache(target).largest_clique())
```

`skgame/optim/embedding.py`, lines 157–164: chain statistics; `n_qubits` feeds the readout-time model

```
157 def chain_stats(embedding: dict) -> dict:
158     """Chain lengths of an embedding: the main predictor of chain breaks and of the energy
159     compression imposed by the chain couplings."""
160     if not embedding:
161         return {"n_qubits": 0, "chain_len_mean": float("nan"), "chain_len_max": 0, "chain_len_min": 0}
162     L = np.array([len(c) for c in embedding.values()])
163     return {"n_qubits": int(L.sum()), "chain_len_mean": float(L.mean()),
164             "chain_len_max": int(L.max()), "chain_len_min": int(L.min())}
```

2.5 `dwave.embedding` (inside `dwave-system`): chains in and out

Capabilities.

- `embed_bqm` maps a logical BQM onto qubits: it splits each J_{ij} over the couplers between two chains and adds $-c$ on the chain couplers;
- `unembed_sampleset` maps reads back to logical spins;
- `chain_strength.uniform_torque_compensation` gives $c = p\, J_{\mathrm{rms}} \sqrt{\bar{d}}$;
- `chain_breaks`: `majority_vote`, `weighted_random`, `discard`, `MinimizeEnergy`.

How we use it.

- The chain strength is `partial(uniform_torque_compensation, prefactor=p)`, with $p =$ `chain_strength_prefactor`.
- The chain-break method is chosen by name.
- `MinimizeEnergy` has to be wrapped in a named function, because `unembed_sampleset` reads `__name__`. Testing revealed this.

`skgame/optim/backends.py`, lines 151–156: the keyword arguments for the physical sampler

```
151     def sample_kwargs(self) -> dict:
152         """Keyword arguments for the (physical) sampler itself."""
153         s = self.settings
154         if self.device == "qpu":
155             return {"num_reads": s.num_reads, "annealing_time": s.annealing_time}
156         return {"num_reads": s.sim_num_reads, "num_sweeps": s.sim_num_sweeps}
```

`skgame/optim/backends.py`, lines 301–311: chain-break resolution by name

```
301 def _chain_break_method(name: str, bqm, emb):
302     from dwave.embedding import chain_breaks as cb
303     if name == "minimize_energy":
304         resolver = cb.MinimizeEnergy(bqm, emb)
305 
306         def minimize_energy(samples, chains):    # unembed_sampleset needs a __name__
307             return resolver(samples, chains)
308         return minimize_energy
309     if name in ("majority_vote", "weighted_random", "discard"):
310         return getattr(cb, name)
311     raise ValueError(f"chain_break_method must be one of {CHAIN_BREAK_METHODS}")
```

2.6 `dwave-samplers` and `dwave-preprocessing`

Capabilities.

- classical samplers with the dimod interface: `SimulatedAnnealingSampler`, `TabuSampler`, `SteepestDescentSolver`, `TreeDecompositionSolver`, `RandomSampler`;
- `SpinReversalTransformComposite` (gauge transforms) and other pre-processing.

How we use it.

- SA, tabu and steepest descent are classical baselines (`solvers.py`).
- SA is also the engine of the `sim:logical` and `sim:*:clique` back-ends.
- Spin-reversal transforms are discussed as a mitigation (`explanation_project.pdf`, noise section) but are not yet wired in.

3 The modes of use

mode	what runs	when to use it	QPU time / token
`sim:logical`	SA on K_N directly (no hardware graph)	reference for the annealing pipeline	none / no
`sim:zephyr:clique`, `sim:pegasus:clique`	the *embedded* problem, chains included, solved by SA on the ideal graph (`StructureComposite`)	test the whole embedding path; classical view of chain effects	none / no

mode	what runs	when to use it	QPU time / token
mock QPU (tests)	`MockDWaveSampler` in place of `DWaveSampler`, same code path as the QPU	tests, dry runs	none / no
`qpu:zephyr:clique`, `qpu:pegasus:clique`	real QPU: `embed_bqm` + `DWaveSampler` + `unembed_sampleset` with our clique embedding	the campaign	yes / yes
`qpu:*:minorminer`	real QPU with a heuristic embedding	baseline only	yes / yes
`QuantumAnnealingSolver`	`DWaveCliqueSampler` (clique, extended J range) or `EmbeddingComposite` (heuristic each call)	the planner benchmark (`skgame/optim/benchmark.py`)	yes / yes

`skgame/optim/quantum.py`, lines 49–58: the single-solver path: `DWaveCliqueSampler` or `EmbeddingComposite`

```
49      def make_sampler(self):
50          if self.use_simulator:
51              from dwave.samplers import SimulatedAnnealingSampler
52              return SimulatedAnnealingSampler()
53          if self.clique:
54              from dwave.system import DWaveCliqueSampler
55              return DWaveCliqueSampler(solver=self.solver_name) if self.solver_name else DWaveCliqueSampler()
56          from dwave.system import DWaveSampler, EmbeddingComposite
57          base = DWaveSampler(solver=self.solver_name) if self.solver_name else DWaveSampler()
58          return EmbeddingComposite(base)
```

The campaign embeds explicitly (the steps of `FixedEmbeddingComposite`) rather than using `DWaveCliqueSampler`, so that both topologies run *identical* code: same embedding strategy, same chain-strength rule, same scaling. Their results then differ only because of the hardware.

4 Life of one QPU call, line by line

`skgame/optim/backends.py`, lines 158–244: `AnnealBackend.sample`: embed, sample, keep the physical reads, unembed

```
158     def sample(self, P: PlannerProblem, rng=None, label: str | None = None) -> dict:
159         """One call. For embedded back-ends this does explicitly what FixedEmbeddingComposite does
160         (embed -> sample -> unembed), so that the PHYSICAL reads are available in between: from
161         them ‘broken_chains‘ tells, read by read, which logical spins had a broken chain."""
162         rng = np.random.default_rng(rng)
163         bqm = P.to_bqm()
164         kw = self.sample_kwargs()
165         params = getattr(self.child, "parameters", {}) or {}
166         if self.device == "sim" and "seed" in params:
167             kw["seed"] = int(rng.integers(2**31))
168         if label and "label" in params:                    # shown in the Leap dashboard next to the problem id
169             kw["label"] = label[:256]
170         broken, phys_counts, cs_used, emb, target = None, None, None, None, None
171         t0 = time.perf_counter()
172         if self.embedded:
173             import dimod
174             from dwave.embedding import EmbeddedStructure, broken_chains, unembed_sampleset
175             from dwave.embedding.chain_strength import uniform_torque_compensation
176             emb = self.embedding(P.N)
177             structure = EmbeddedStructure(self.child.edgelist, emb)
178             target = structure.embed_bqm(bqm, chain_strength=partial(
179                 uniform_torque_compensation, prefactor=self.settings.chain_strength_prefactor),
180                 smear_vartype=dimod.SPIN)
181             cs_used = structure.chain_strength
182             to_send, scaling = self._scale(target, emb, float(cs_used))
183             if scaling["mode"] != "auto_scale":
184                 kw["auto_scale"] = False
185             if self.flux_biases and "flux_biases" in params:
```

```
            nq = int((getattr(self.child, "properties", {}) or {}).get("num_qubits", 0)) or \
                (max(map(int, self.child.nodelist)) + 1)
            fb = [0.0] * nq
            for q, v in self.flux_biases.items():
                fb[int(q)] = float(v)
            kw["flux_biases"] = fb
            if "flux_drift_compensation" in params:
                kw["flux_drift_compensation"] = True
        phys = self.child.sample(to_send, **kw)
        phys.resolve()
        chains = [emb[i] for i in range(P.N)]
        broken = np.asarray(broken_chains(phys, chains), bool)          # (physical rows, N)
        phys_counts = np.asarray(phys.record.num_occurrences, int)
        ss = unembed_sampleset(phys, structure, bqm,
                               chain_break_method=_chain_break_method(self.settings.chain_break_method, bqm,
 emb),
                               chain_break_fraction=True)
        ss.info.update({k: v for k, v in phys.info.items() if k not in ss.info})
    else:
        ss = self.child.sample(bqm, **kw)
        ss.resolve()
    wall = time.perf_counter() - t0
    out = sampleset_statistics(ss, P, wall, polish=self.settings.polish, broken=broken, phys_counts=
phys_counts,
                               discard=self.settings.chain_break_method == "discard")
    info = out["info"]
    info.update(backend=self.spec, solver=self.solver_id(), settings=asdict(self.settings),
                sample_kwargs={k: v for k, v in kw.items() if k not in ("seed", "flux_biases")})
    if "flux_biases" in kw:
        v = np.array([x for x in kw["flux_biases"] if x != 0.0])
        info["flux_biases"] = {"source": self.flux_biases_source, "n_nonzero": int(v.size),
                               "rms": float(np.sqrt(np.mean(v ** 2))) if v.size else 0.0}
    if self.record_qubit_magnetization and self.embedded:
        w = np.asarray(phys.record.num_occurrences, float)
        m = (np.asarray(phys.record.sample, float) * w[:, None]).sum(0) / w.sum()
        info["qubit_magnetization"] = {str(int(q)): float(x) for q, x in zip(phys.variables, m)}
    for k in ("problem_id", "problem_label"):
        if k in ss.info:
            info[k] = str(ss.info[k])
    info["solver_properties"] = self.solver_snapshot()
    if emb is not None:
        info["embedding"] = chain_stats(emb)
        info["embedding_used"] = {str(i): [int(q) for q in emb[i]] for i in range(P.N)}
        info["chain_lengths"] = [len(emb[i]) for i in range(P.N)]
        info["chain_strength"] = float(cs_used) if np.isscalar(cs_used) else None
        info["chain_strength_prefactor"] = float(self.settings.chain_strength_prefactor)
        info["max_abs_logical_coupling"] = float(np.max(np.abs(P.W)))
        q = np.array(list(target.quadratic.values()), float)
        lo, hi = (info["solver_properties"].get("j_range") or [-1.0, 1.0])
        # what auto_scale does here (no linear terms): divide all couplers by the factor that
        # brings the most extreme one to the end of the allowed range [lo, hi]
        worst = max(q.max(initial=0.0) / hi, q.min(initial=0.0) / lo)
        uv = np.array(list(target.quadratic.keys()), dtype=np.int64)
        out["_physical"] = {"u": uv[:, 0], "v": uv[:, 1], "J": q,
                            "h_q": np.array(list(target.linear.keys()), dtype=np.int64),
                            "h": np.array(list(target.linear.values()), float), "offset": float(target.
offset)}
        info["programmed"] = {"max_abs_J_before_scaling": float(np.abs(q).max()),
                              "auto_scale_divisor": float(worst) if scaling["mode"] == "auto_scale"
                              else float(scaling["divisor"]), "scaling": scaling,
                              "n_qubits": int(target.num_variables), "n_couplers": int(target.
num_interactions)}
    return out
```

What happens, in order:

1. `P.to_bqm()`: the SK problem as an Ising BQM.
2. `self.embedding(P.N)`: the clique embedding of K_N on the solver's real graph, read from `embeddings/<solver>/clique/N....json` or computed and cached.
3. `EmbeddedStructure(child.edgelist, emb).embed_bqm(bqm, chain_strength=...)` computes c with the chain-strength callable, splits each J_{ij} over the couplers between two chains and puts $-c$ on the chain couplers.

4. `DWaveSampler.sample(target, num_reads, annealing_time)` submits the physical problem to SAPI. `auto_scale` is on by default and divides by the largest coefficient.

5. `broken_chains(phys, chains)`: for every physical read and every logical spin, whether its chain is broken (qubits disagreeing).

6. `unembed_sampleset(phys, structure, bqm, chain_break_method=..., chain_break_fraction=True)` maps the reads back to logical spins.

7. `ss.resolve()` waits for the asynchronous answer.

8. `sampleset_statistics` recomputes every reward from the spins: it is exact whatever scaling was applied. It aggregates identical reads, records `info["timing"]` (in μs) and the read-averaged chain-break fraction, and optionally polishes every distinct read with greedy descent.

9. The embedding statistics, the chain strength actually used (from `embedding_context`) and $\max|W_{ij}|$ are added.

`skgame/optim/backends.py`, lines 314–386: statistics of a sample set, including the broken chains of every read

```
def sampleset_statistics(ss, P: PlannerProblem, wall: float, polish: bool = False, broken=None,
                         phys_counts=None, discard: bool = False) -> dict:
    """Distinct rewards and their read counts (best first), best spins, QPU timing, chain breaks.

    Rewards are recomputed from the spins, so they are exact in units of R_N whatever scaling
    the sampler applied. With ‘polish‘, every distinct read is also relaxed by greedy single-flip
    ascent (a classical post-processing step that repairs the residual errors of chain breaks
    and of analogue noise) and reported separately.

    With ‘broken‘ (physical rows x N booleans from dwave.embedding.broken_chains) the result also
    has, read by read, how many chains broke:
      out["reads"]                 distinct (configuration, number of broken chains) pairs:
                                   reward, polished_reward, n_broken, count
      info["broken_chains_hist"]   {number of broken chains: number of reads}
      info["chain_break_count"]    for each logical spin i, in how many reads its chain broke
      info["discarded_reads"]      reads dropped by chain_break_method="discard"
    """
    agg = ss.aggregate()
    if len(agg) == 0:                       # chain_break_method="discard" dropped every read
        S = np.zeros((0, P.N))
    else:
        cols = [list(agg.variables).index(i) for i in range(P.N)]
        S = np.asarray(agg.record.sample, float)[:, cols]
    counts = np.asarray(agg.record.num_occurrences, int)
    rewards = np.atleast_1d(P.reward(S)) if len(S) else np.zeros(0)
    order = np.argsort(-rewards)
    info = {"wall_seconds": wall, "num_reads": int(counts.sum())}
    timing = ss.info.get("timing")
    if timing:
        info["timing_us"] = {k: float(v) for k, v in timing.items() if np.isscalar(v)}
    if "chain_break_fraction" in ss.record.dtype.names and len(ss):
        info["chain_break_fraction_mean"] = float(np.average(ss.record.chain_break_fraction,
                                                             weights=ss.record.num_occurrences))
    empty = len(S) == 0
    out = {"rewards": rewards[order].tolist() if not empty else [], "counts": counts[order].tolist(),
           "best_reward": float(rewards[order[0]]) if not empty else -np.inf,
           "best_S": _spin_string(S[order[0]]) if not empty else "", "info": info}
    polish_of = None
    if polish and empty:
        out.update(polished_rewards=[], polished_counts=[], polished_best_reward=-np.inf, polished_best_S="")
        polish_of = {}
    elif polish:
        from .solvers import _greedy
        Sp = np.array([_greedy(P.W, s.copy())[0] for s in S])
        rp = np.atleast_1d(P.reward(Sp))
        op = np.argsort(-rp)
        out.update(polished_rewards=rp[op].tolist(), polished_counts=counts[op].tolist(),
                   polished_best_reward=float(rp[op[0]]), polished_best_S=_spin_string(Sp[op[0]]))
        polish_of = {S[k].tobytes(): float(rp[k]) for k in range(len(S))}
    if broken is not None:
        nb_phys = broken.sum(axis=1)
```

```
        info["chain_break_count"] = (broken * phys_counts[:, None]).sum(axis=0).astype(int).tolist()
        hist = {}
        for nb, c in zip(nb_phys, phys_counts):
            hist[int(nb)] = hist.get(int(nb), 0) + int(c)
        info["broken_chains_hist"] = {str(k): v for k, v in sorted(hist.items())}
        kept = nb_phys == 0 if discard else np.ones(len(nb_phys), bool)
        info["discarded_reads"] = int(phys_counts[~kept].sum())
        nb_rows = nb_phys[kept]
        Sr = (np.asarray(ss.record.sample, float)[:, [list(ss.variables).index(i) for i in range(P.N)]]
              if len(ss) else np.zeros((0, P.N)))
        assert len(Sr) == len(nb_rows), "unembedded rows do not match the physical reads"
        table = {}
        for s_row, nb, c in zip(Sr, nb_rows, ss.record.num_occurrences):
            key = (s_row.tobytes(), int(nb))
            table[key] = table.get(key, 0) + int(c)
        keys = list(table)
        Sk = np.array([np.frombuffer(k[0]) for k in keys]).reshape(len(keys), P.N)
        rk = np.atleast_1d(P.reward(Sk)) if keys else np.zeros(0)
        out["reads"] = {"S": [_spin_string(x) for x in Sk], "reward": rk.tolist(), "n_broken": [k[1] for k in
    keys],
                        "count": [table[k] for k in keys],
                        "polished_reward": [polish_of[k[0]] for k in keys] if polish_of is not None else None}
    return out
```

The `seed` lesson. Samplers differ in which arguments they accept. The QPU rejects unknown ones *server-side*, so a Python `try/except TypeError` would never catch the error. Both wrappers therefore pass `seed` only to samplers that declare it in `parameters`:

`skgame/optim/solvers.py`, lines 379–388: passing `seed` only where it is accepted

```
        # only samplers that declare a 'seed' parameter get one: the QPU rejects unknown kwargs
        # server side (not with a TypeError), so trying and catching is not enough there
        params = getattr(self._sampler, "parameters", None)
        if params is None or "seed" in params:
            kw.setdefault("seed", int(rng.integers(2**31)))
        try:
            ss = self._sampler.sample(bqm, **kw)
        except TypeError:
            kw.pop("seed", None)
            ss = self._sampler.sample(bqm, **kw)
```

5 What `qpu --check` does (and why it costs nothing)

`python main_simulation.py qpu --out DIR --check` runs, for each QPU back-end of `config.json`:

1. `make_backend` builds an `AnnealBackend`.
2. `be.largest_problem()` builds `DWaveSampler(solver={"topology__type": ...})`. This contacts SAPI to choose an online solver of that family and download its properties: **metadata only, no problem is submitted**. The method then reads the working graph and calls `busgraph_cache(G).largest_clique()`.
3. The list of calls is built. N above the largest clique is skipped; already-saved results are skipped (resume).
4. `_estimate_qpu_time` computes, for each N, the clique embedding on the real graph and the number of qubits it uses, then evaluates `solver.estimate_qpu_access_time`: **a local formula, no QPU**.

`main_simulation.py`, lines 479–492: the `--check` estimate

```
def _estimate_qpu_time(be, todo, log) -> None:
    """QPU access time still to spend, from the solver's own timing model (no QPU time used)."""
    if be.device != "qpu":
```

```
        log("[anneal] not a QPU back-end: nothing to estimate")
        return
    per_N = {}
    for N in sorted({u[3] for u in todo}):
        per_N[N] = be.estimate_access_time(N)
        if per_N[N] is None:
            log("[anneal] the solver exposes no timing model")
            return
        log(f"[anneal]   N={N:4d}: ~{per_N[N] * 1e3:.0f} ms per call")
    total = sum(per_N[u[3]] for u in todo)
    log(f"[anneal] estimated QPU access time for the {len(todo)} remaining calls: {total:.1f} s")
```

`skgame/optim/backends.py`, lines 291–298: one call's estimate from the solver's timing model

```
    def estimate_access_time(self, N: int) -> float | None:
        """Seconds of QPU access time for one call, from the solver's timing model (no QPU use)."""
        solver = getattr(self.child, "solver", None)
        est = getattr(solver, "estimate_qpu_access_time", None)
        if est is None:
            return None
        n_qubits = chain_stats(self.embedding(N))["n_qubits"]
        return est(n_qubits, num_reads=self.settings.num_reads, annealing_time=self.settings.annealing_time) / 1
     e6
```

back-end	solver	largest N	calls (`full`)	estimated time
`qpu:zephyr:clique`	`Advantage2_system1`	104	260 ($N \leq 104$)	52.3 s
`qpu:pegasus:clique`	`Advantage_system4`	177	400	105.0 s

Your check (27 October). On the real Zephyr chip the largest all-to-all problem is $N = 104$, far below the 184 of the ideal graph. Defects remove intact line segments (`explanation_project.pdf`, embedding step 10).

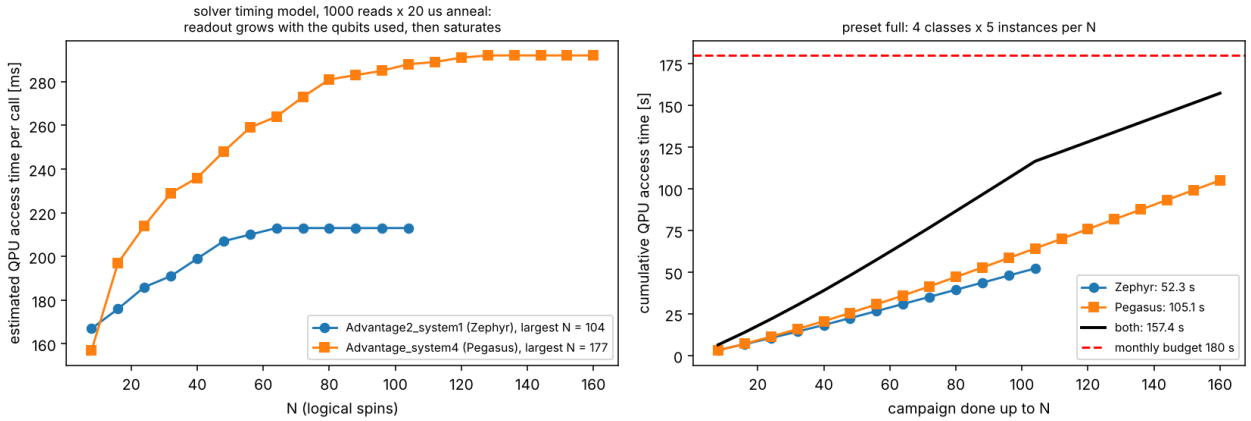


Figure 2: Left: estimated access time per call, from your `--check`. Right: cumulative budget of the `full` campaign: 52.3 s (Zephyr) + 105.0 s (Pegasus) = 157.3 s, against 180 s.

6 How QPU time is estimated: three levels

6.1 Level 1: the rough plan (`main_simulation.py plan`)

For a first idea, before any connection:

$$t_{\text{call}} \approx t_{\text{prog}} + n_{\text{reads}}\,(t_a + t_{\text{ro}} + t_d) = 15\,\text{ms} + 1000\,(20 + 100 + 20)\,\mu\text{s} = 155\,\text{ms}. \tag{1}$$

The constants are generic, and your check shows they **underestimate**: the real per-call times are 157–292 ms. Use level 1 only to compare designs, never to spend.

6.2 Level 2: the solver's timing model (`--check`, budget guard)

D-Wave publishes, in each solver's `properties["problem_timing_data"]`, the parameters of a model that `estimate_qpu_access_time` evaluates. Reading its source (`dwave-cloud-client` 0.14.8) gives

$$t_{\text{access}} = \underbrace{t_{\text{prog}}^{\text{typ}} + t_{\text{prog}}^{\text{therm}}}_{\text{programming}} + n_{\text{reads}}\Big(t_a + t_{\text{ro}}(n_{\text{qubits}}) + t_{\text{delay}} + \max(t_{\text{ro}}^{\text{therm}}, t_{\text{decorr}})\Big), \tag{2}$$

$$t_{\text{ro}}(n_{\text{qubits}}) = \text{piecewise-linear interpolation (often in log–log) of a measured curve.} \tag{3}$$

Two consequences, both visible in your check:

- the cost grows with N, because more qubits mean a longer readout, and then **saturates**;
- Pegasus costs more per call than Zephyr (292 vs 213 ms at large N), because its chains use more qubits.

Above all, the time is **proportional to `num_reads`** apart from the programming term. Halving the reads roughly halves the budget.

6.3 Level 3: the charge (`qpu_access_time`)

After each call the QPU returns its timing. `qpu_access_time` is what Leap charges. It is saved in every result file together with the level-2 estimate (`estimated_access_s`), and `qpu --spent` sums them and prints the ratio actual/estimate:

`main_simulation.py`, lines 461–476: `qpu --spent`: what was actually charged

```
461 def qpu_spent(out: Path, log=print) -> dict:
462     """QPU access time already spent in a campaign, per back-end, from the saved results
463     (the value D-Wave charged for each call), with the ratio to the prediction."""
464     rows = {}
465     for f in sorted((out / "anneal").glob("qpu_*/*/*.json")) if (out / "anneal").exists() else []:
466         info = json.loads(f.read_text())["info"]
467         r = rows.setdefault(f.parent.parent.name, {"calls": 0, "spent_s": 0.0, "est_s": 0.0})
468         r["calls"] += 1
469         r["spent_s"] += info.get("timing_us", {}).get("qpu_access_time", 0.0) / 1e6
470         r["est_s"] += info.get("estimated_access_s", 0.0)
471     total = sum(r["spent_s"] for r in rows.values())
472     for name, r in rows.items():
473         ratio = f"{r['spent_s'] / r['est_s']:.2f}" if r["est_s"] else "n/a"
474         log(f"[spent] {name:22s} {r['calls']:4d} calls  {r['spent_s']:8.3f} s  (actual/estimate = {ratio})")
475     log(f"[spent] total QPU access time in {out}: {total:.3f} s")
476     return rows
```

7 The budget guard: `qpu --budget-seconds S`

`main_simulation.py`, lines 355–386: budget object and per-call estimate

```
355 class QPUBudget:
356     """Hard cap on the QPU access time spent by one command (all back-ends together).
357 
358     Before each call the cost is predicted with the solver's own timing model
359     (‘estimate_qpu_access_time‘) and inflated by a safety factor; the call is made only if
360     spent + inflated estimate <= limit. The factor starts at ‘safety‘ and grows if real
361     charges come out larger than predicted (factor >= 1.1 x worst observed actual/estimate).
362     After each call the REAL ‘qpu_access_time‘ returned by the QPU is added to ‘spent‘.
363     """
364 
365     def __init__(self, limit_s: float | None, safety: float = 1.5):
366         self.limit, self.safety, self.spent, self.worst_ratio = limit_s, safety, 0.0, 0.0
367 
368     def factor(self) -> float:
369         return max(self.safety, 1.1 * self.worst_ratio)
370 
```

```
    def allows(self, estimate_s: float) -> bool:
        return self.limit is None or self.spent + self.factor() * estimate_s <= self.limit

    def charge(self, actual_s: float, estimate_s: float) -> None:
        self.spent += actual_s
        if estimate_s > 0 and actual_s > 0:
            self.worst_ratio = max(self.worst_ratio, actual_s / estimate_s)


def _call_estimate(be, N: int, cfg: CampaignConfig) -> float:
    """Predicted QPU access time of one call [s]. Without a timing model (mock QPU) a
    deliberately pessimistic formula: 20 ms programming + reads x (anneal + 300 us)."""
    est = be.estimate_access_time(N) if be.device == "qpu" else 0.0
    if est is None:
        est = 0.020 + cfg.num_reads * (cfg.annealing_time * 1e-6 + 300e-6)
    return float(est)
```

main_simulation.py, lines 401–458: the anneal stage: guard before every call, real charge after it

```
def stage_anneal(out: Path, cfg: CampaignConfig, spec: str = "sim:logical", keys=None, max_calls=None,
                 solver_name=None, check_only=False, log=print, retries=3, backend=None,
                 budget: QPUBudget | None = None, Ns=None) -> None:
    be = backend or make_backend(out, cfg, spec, solver_name)
    limit = be.largest_problem()
    log(f"[anneal] backend={be.spec} solver={be.solver_id()} largest_problem={limit}")
    units = list(anneal_units(out, cfg, keys, Ns=Ns))
    too_big = sorted({u[3] for u in units if limit is not None and u[3] > limit})
    if too_big:
        log(f"[anneal] skipping N={too_big}: above the largest embeddable clique ({limit})")
    todo = [u for u in units if (limit is None or u[3] <= limit)
            and not anneal_path(out, be.label, u[0], u[3], u[4]).exists()]
    log(f"[anneal] {len(units)} calls in the campaign, {len(todo)} still to do")
    if check_only:
        _estimate_qpu_time(be, todo, log)
        return
    total_access = 0.0
    budget = budget or QPUBudget(None)
    for n, (key, eps, J0, N, b) in enumerate(todo):
        if max_calls is not None and n >= max_calls:
            log(f"[anneal] stopped after --max-calls {max_calls}; rerun to continue")
            break
        est = _call_estimate(be, N, cfg)
        if not budget.allows(est):
            log(f"[anneal] budget: spent {budget.spent:.3f}s of {budget.limit:.3f}s; next call (N={N}) estimated
  "
                f"{est:.3f}s x safety {budget.factor():.2f} would exceed it -> stopped (rerun to continue)")
            break
        J = ensure_instances(out, cfg, key, eps, J0, N)
        P = PlannerProblem(J[b])
        if cfg.chain_strength_by_N or cfg.chain_strength_by_class:   # p chosen by calibration / coupling-scan
            from dwave_utils.coupling_scan import prefactor_for
            be.settings.chain_strength_prefactor = prefactor_for(cfg, be.label, key, N)
        for attempt in range(retries + 1):
            try:
                res = be.sample(P, rng=np.random.SeedSequence([cfg.seed, 3, N, b]),
                                label=f"skgame {out.name} {be.label} {key} N={N} i={b}")
                break
            except Exception as exc:                        # network hiccups, solver busy, ...
                if attempt == retries or not _transient(exc):
                    raise
                log(f"[anneal] {key} N={N} i={b}: {type(exc).__name__}: {exc}; retry in {2 ** (attempt + 1)}s")
                time.sleep(2 ** (attempt + 1))
        res.update(key=key, eps=eps, J0=J0, N=N, instance=b, backend=be.spec)
        res["info"]["estimated_access_s"] = est
        phys = res.pop("_physical", None)
        if phys is not None:                        # the exact physical problem sent (before auto_scale)
            pp = anneal_path(out, be.label, key, N, b).with_name(f"N{N:04d}_i{b}_physical.npz")
            pp.parent.mkdir(parents=True, exist_ok=True)
            np.savez_compressed(pp, **phys)
            res["info"]["physical_problem_file"] = pp.name
        _write_json(anneal_path(out, be.label, key, N, b), res)
        acc = res["info"].get("timing_us", {}).get("qpu_access_time", 0.0) / 1e6
        budget.charge(acc if acc > 0 else est, est)
        total_access += acc
        cbf = res["info"].get("chain_break_fraction_mean")
```

```
456         log(f"[anneal] {n + 1:4d}/{len(todo)} {be.label} {key:10s} N={N:4d} i={b} best={res['best_reward']:.5f}
    "
457             f"chain_breaks={'-' if cbf is None else f'{cbf:.3f}'} qpu_access={acc * 1e3:.1f}ms (est {est * 1e3
    :.0f}ms) "
458             f"(total {total_access:.3f}s)")
```

Rule: before each call, with $\hat{t}$ the level-2 estimate,

$$\text{call allowed} \iff t_{\text{spent}} + f\,\hat{t} \le S, \qquad f = \max\big(1.5,\ 1.1 \cdot \max_{\text{past calls}} t_{\text{actual}}/\hat{t}\big). \tag{4}$$

- The guard starts with a 50% safety margin. If the real charges turn out larger than the model, the margin grows automatically.
- After every call t_{spent} grows by the *real* `qpu_access_time`.
- The budget covers all back-ends of one command (Zephyr, then Pegasus).
- When a call is refused, the command stops cleanly. Everything done so far is saved, and the same command continues later (resume).
- Without a timing model (mock QPU) the estimate is a deliberately pessimistic formula: 20 ms+ $n_{\text{reads}}(t_a + 300\,\mu\text{s})$.

It is tested in `tests/test_campaign.py`: the rule itself, a 0.01 s budget that refuses every call, and a 1 s budget that completes and records estimates.

What the guard cannot guarantee. It caps the time *this code* spends, using D-Wave's own numbers. It cannot stop a call once submitted, and it does not see time spent by other programs or other users of the same Leap project. The Leap dashboard remains the authority on what is left.

8 Broken chains, read by read

What is saved. For every call, `sampleset_statistics` (listing above) stores:

- `reads`: one row per distinct pair (configuration, number of broken chains), with the reward, the reward after greedy polishing, the number of broken chains and the number of reads;
- `info.broken_chains_hist`: number of reads with $0, 1, 2, \ldots$ broken chains;
- `info.chain_break_count`: for each logical spin i, in how many reads its chain broke. With equal-length clique chains this should be roughly uniform; outliers point at faulty qubits or couplers;
- `info.chain_lengths`, `info.chain_strength`, `info.discarded_reads` (only with `chain_break_method=discard`).

A chain of spin i is *broken* in a read when its qubits do not all agree. Its logical value is then decided by the chain-break method (majority vote by default). The fraction D-Wave reports, `chain_break_fraction`, is exactly (broken chains)/N, and the code checks this.

Analysis. `python main_simulation.py chains --out DIR` (also run by `analyze`, and usable before the classical stage) writes `summary/chain_breaks.csv` and `chain_breaks.png`. For each back-end, N and number of broken chains k they give the fraction of reads, the success probability, and the mean residual, raw and after polishing. The reference is the exact or classical optimum when available, otherwise the best read seen.

`main_simulation.py`, lines 739–794: solution quality vs number of broken chains

```python
def chain_break_table(out: Path, cfg: CampaignConfig) -> list[dict]:
    """Solution quality as a function of the number of broken chains in the read.

    One row per (back-end, problem class, N, number of broken chains): how many reads, and among
    them the success probability and the mean residual, raw and after greedy polishing. The
    reference is the same as in `collect` (exact / classical, improved by any annealer read);
    if the classical stage has not run yet it is the best read of all annealers ("best seen").
    """
    if not (out / "anneal").exists():
        return []
    labels = sorted(p.name for p in (out / "anneal").glob("*"))
    rows = []
    for key, eps, J0 in _classes(cfg):
        for N in cfg.Ns:
            recs = {lab: [json.loads(p.read_text()) if p.exists() else None
                          for p in (anneal_path(out, lab, key, N, b) for b in range(cfg.n_instances))]
                    for lab in labels}
            if not any(r for v in recs.values() for r in v):
                continue
            cp = classical_path(out, key, N)
            if cp.exists():
                c = json.loads(cp.read_text())
                ref, src = np.array(c["ref"], float), ("exact" if c["ref_exact"] else "classical")
            else:
                ref, src = np.full(cfg.n_instances, -INF), "best_seen"
            for v in recs.values():
                for b, r in enumerate(v):
                    if r is not None:
                        ref[b] = max(ref[b], r["best_reward"], r.get("polished_best_reward", -INF))
            for lab, v in recs.items():
                acc = {}
                for b, r in enumerate(v):
                    if r is None or "reads" not in r:
                        continue
                    t = r["reads"]
                    rw, nb, ct = (np.array(t[k], float) for k in ("reward", "n_broken", "count"))
                    pr = np.array(t["polished_reward"], float) if t.get("polished_reward") else None
                    ok = _success(rw, ref[b])
                    okp = _success(pr, ref[b]) if pr is not None else None
                    for k in np.unique(nb).astype(int):
                        m = nb == k
                        a = acc.setdefault(k, {"reads": 0, "succ": 0.0, "succ_p": 0.0, "res": 0.0, "res_p":
    0.0})
                        a["reads"] += ct[m].sum()
                        a["succ"] += ct[m][ok[m]].sum()
                        a["res"] += np.sum(ct[m] * (ref[b] - rw[m]) / abs(ref[b]))
                        if pr is not None:
                            a["succ_p"] += ct[m][okp[m]].sum()
                            a["res_p"] += np.sum(ct[m] * (ref[b] - pr[m]) / abs(ref[b]))
                tot = sum(a["reads"] for a in acc.values())
                for k, a in sorted(acc.items()):
                    rows.append({"backend": lab, "problem": key, "N": N, "n_broken": k, "reads": int(a["reads"])
    ,
                                 "frac_reads": a["reads"] / tot, "p_success": a["succ"] / a["reads"],
                                 "p_success_polished": a["succ_p"] / a["reads"],
                                 "residual_mean": a["res"] / a["reads"], "residual_polished_mean": a["res_p"] /
    a["reads"],
                                 "reference": src})
    return rows
```

Example (classical simulator, deliberately weak chains $p = 0.5$). At $N = 16$ on Pegasus, 70% of the reads have no broken chain and hit the optimum in 88% of cases. Reads with one or more broken chains never do, raw. Greedy polishing brings those with 1–2 broken chains back to 84–96%. The residual grows almost linearly with the number of broken chains. On the real QPU, with the default $p = 1.414$, the test of 27 October had chain-break fractions of 0 and 0.001 at $N \leq 16$.

9 Ground states

`python main_simulation.py ground --out DIR` (also run by `analyze`) writes `summary/ground_states.csv`. For every instance it gives:

- the best configuration found (a string of $+/-$, one per agent), with the sign fixed so that agent 0 is $+$, since S and $-S$ have the same energy;
- its reward R_N and its energy $H = -\frac{N}{2}(R_N - \text{offset})$;
- where it came from, and whether it is **certified** (exact enumeration, $N \leq 24$) or only **best known**;
- for every source (classical reference, each annealer raw and polished) whether that source found it, and its Hamming distance from it.

The classical stage now saves the reference *configuration* (`ref_S`, `ref_source`) and not only its value. Each QPU result file already holds `best_S` and `polished_best_S`.

`main_simulation.py`, lines 866–928: the ground-state report

```
866 def ground_states(out: Path, cfg: CampaignConfig, log=print) -> list[dict]:
867     """The ground state (best configuration) of every instance, from every source available.
868 
869     Candidates: the classical reference (exact Gray-code enumeration for N <= n_exact: certified;
870     otherwise the best classical solver / long parallel tempering: best known) and, for every
871     annealing back-end, its best read and its best polished read. The best candidate is reported
872     with its reward R_N, energy H = -N (R_N - offset)/2, configuration (+/- per agent, sign fixed
873     so that agent 0 is +, since S and -S are equivalent), where it came from, whether it is
874     certified, and for each source whether it found it and how far (Hamming distance) its best
875     configuration is. Writes summary/ground_states.csv."""
876     labels = sorted(p.name for p in (out / "anneal").glob("*")) if (out / "anneal").exists() else []
877     rows = []
878     for key, eps, J0 in _classes(cfg):
879         for N in cfg.Ns:
880             cp = classical_path(out, key, N)
881             c = json.loads(cp.read_text()) if cp.exists() else None
882             if c is None and not any(anneal_path(out, l, key, N, 0).exists() for l in labels):
883                 continue
884             J = ensure_instances(out, cfg, key, eps, J0, N)
885             for b in range(cfg.n_instances):
886                 P = PlannerProblem(J[b])
887                 cand = []                                   # (reward, spins, column, source)
888                 if c is not None and c.get("ref_S"):
889                     cand.append((c["ref"][b], c["ref_S"][b], "classical", c["ref_source"][b]))
890                 for lab in labels:
891                     ap = anneal_path(out, lab, key, N, b)
892                     if ap.exists():
893                         r = json.loads(ap.read_text())
894                         if r.get("best_S"):
895                             cand.append((r["best_reward"], r["best_S"], lab, lab))
896                         if r.get("polished_best_S"):
897                             cand.append((r["polished_best_reward"], r["polished_best_S"], lab + "+polish", lab +
      "+polish"))
898                 if not cand:
899                     continue
900                 top = max(t[0] for t in cand)               # ties (to rounding) go to the first source:
     classical/exact
901                 best = next(t for t in cand if t[0] >= top - TOL * max(1, abs(top)))
902                 g = _canonical(best[1])
903                 S = np.array([1.0 if ch == "+" else -1.0 for ch in g])
904                 certified = bool(c is not None and c.get("ref_exact") and c.get("ref_S"))
905                 row = {"problem": key, "N": N, "instance": b, "reward": float(P.reward(S)),
906                        "energy": float(P.energy(S)), "ground_state": g, "found_by": best[3],
907                        "certified": certified, "status": "exact optimum" if certified else "best known"}
908                 for rw, sp, col, _src in cand:
909                     row[f"{col}:found"] = bool(rw >= top - TOL * max(1, abs(top)))
910                     row[f"{col}:hamming"] = sum(x != y for x, y in zip(_canonical(sp), g))
911                 rows.append(row)
912     if rows:
913         path = out / "summary" / "ground_states.csv"
914         path.parent.mkdir(parents=True, exist_ok=True)
```

```
        cols = []
        for r in rows:
            cols += [k for k in r if k not in cols]
        with open(path, "w", newline="") as f:
            w = csv.DictWriter(f, fieldnames=cols)
            w.writeheader()
            w.writerows(rows)
        for r in rows:
            log(f"[ground] {r['problem']:10s} N={r['N']:4d} i={r['instance']} R_N={r['reward']:.5f} "
                f"H={r['energy']:9.4f} {r['status']:13s} by {r['found_by']:24s} {r['ground_state'] if r['N'] <=
 40 else r['ground_state'][:40] + '...'}")
        log(f"[ground] -> {path}")
    else:
        log("[ground] no results yet")
    return rows
```

When the QPU's graph changes. A QPU's working graph changes over time as qubits and couplers go offline or come back. On 2 November the Zephyr chip's largest clique was 105, against 104 on 27 October. An embedding cached on an earlier day may then contain a chain that is no longer connected (`DisconnectedChainError`). `EmbeddingCache.get` therefore re-validates every cached embedding against the current graph (`is_valid_embedding`), recomputes it if it no longer fits, and keeps the old file as `N....stale.json`. Errors in the embedding or the problem are no longer retried (`_transient`), and they occur before any QPU time is used.

10 Chain strength and coupling scale: what is set, and what is not known

What is set. The chain strength is $c = p\,J_{\mathrm{rms}}\sqrt{N-1} \approx p$ for SK, with the prefactor p (`chain_strength_prefactor`, default 1.414, overridden per N by `coupling-scan`) the only free knob. Why p must be larger for the all-to-all problem: `prefactor_and_shimming.pdf`. The logical couplings W_{ij} are split evenly over the $|E_{ij}|$ couplers between chains (about 1.1–1.2), and `auto_scale` divides the whole physical problem by its largest $|J|$. With the default p that is the chain coupling c, so the SK couplings end up at about 0.05 on $[-1, 1]$. With a small p (e.g. 0.5) the largest coupling can be a logical one, as the tests show, and then the logical couplings are no longer the ones compressed by c.

What is not known in advance. There is no formula for the best p:

- too small, and chains break: their spins are guessed by the chain-break method;
- too large, and chains are rigid and the SK part is squeezed towards the analogue noise.

The optimum depends on N, on the instance class and on the chip. The same holds for the chain-break method and the annealing time. The default is a well-founded starting point, not an optimum. On the QPU it can only be found empirically: run a few values of p on the same instances and compare the success rate together with the broken-chain statistics. `scan-summary` does this comparison across campaign directories that differ only in p (README_LOCAL.md gives the commands).

What is recorded to judge it. For every call:

- c and the auto-scale divisor;
- the per-read broken chains and which chains break;
- the exact physical problem sent (couplers and values, `*_physical.npz`).

A rule of thumb from D-Wave's guidance: chain-break fractions of a few per cent or less, with the success rate still rising as p decreases, mean there is room to lower p.

11 Choosing the chain strength for every N: `coupling-scan`

For every QPU and every N of a (reduced) grid, the same instance is annealed with each prefactor p of a grid, by default $0.75, 1, 1.414, 2, 3, 4, 6$, with 200 reads. The grid reaches 6 because the chains of the all-to-all SK problem need $p \approx 3$–4 at $N \gtrsim 30$: the derivation, the upper bound set by the control error, the extended coupler range and the flux-bias shimming (`shim`) are in `prefactor_and_shimming.pdf`. The scan counts the broken chains of every read.

The rule. Take the *smallest* p whose mean fraction of broken chains is at most a threshold (default 0: no chain broken in any read). If no p reaches it, take the p with the fewest broken chains. Minimising the breaks alone would always select the largest p tested, since breaks decrease monotonically with p, while a larger p compresses the SK couplings further (auto-scale divides by $\approx c$). The smallest p that keeps the chains intact is the standard compromise. The output flags a choice at the edge of the grid.

`dwave_utils/coupling_scan.py`, lines 36–49: the choice of p

```
36 def choose_coupling(rows: list[dict], threshold: float = 0.0) -> tuple[float, str]:
37     """rows: [{"p", "broken_fraction"}...] for ONE back-end and N. Returns (p, reason)."""
38     ok = [r for r in rows if r["broken_fraction"] <= threshold + 1e-15]
39     if ok:
40         best = min(ok, key=lambda r: r["p"])
41         why = f"smallest p with broken-chain fraction <= {threshold:g}"
42         if best["p"] == min(r["p"] for r in rows):
43             why += " (already at the SMALLEST p tested: a smaller p may also work)"
44         return best["p"], why
45     best = min(rows, key=lambda r: (r["broken_fraction"], r["p"]))
46     why = "no p reached the threshold: fewest broken chains"
47     if best["p"] == max(r["p"] for r in rows):
48         why += " (at the LARGEST p tested: extend the grid upwards)"
49     return best["p"], why
```

Use in the main run. The table $\{\text{QPU} : \{N : p\}\}$ is stored in `config.json` (`chain_strength_by_N`). The QPU stage sets p call by call: exact for scanned N, linear interpolation in between, the last value beyond the largest scanned N. Every result records the p actually used, and `validate` checks the chain strength of each call against its own p.

`dwave_utils/coupling_scan.py`, lines 52–60: prefactor for a given N

```
52 def coupling_for(table: dict, label: str, N: int, default: float) -> float:
53     """Prefactor for back-end `label` at size N from a {label: {N: p}} table: exact N, else linear
54     interpolation between the nearest scanned N, else the nearest end (constant extrapolation)."""
55     t = table.get(label) or {}
56     if not t:
57         return default
58     xs = sorted(int(k) for k in t)
59     ys = [float(t[str(x)] if str(x) in t else t[x]) for x in xs]
60     return float(np.interp(N, xs, ys))
```

Cost. The scan costs ≈ 5.4 s of QPU time for $N = 8, \ldots, 48$ and 104 on both QPUs. That is about 15.8 ms of programming plus 200 reads per call, for 98 calls. The time does not depend on p.

12 The QPU pipeline: `dwave_utils`

All the D-Wave tools are in the package `dwave_utils`. Each command can be launched as `python -m dwave_utils <command>` or `python main_simulation.py <command>`, and each one is also a Python function (`import dwave_utils as du`). The pipeline is linear:

`feasibility` → `init` → `budget` → `calibrate` → `budget --fill` → `run` → `report` → (`classical`) → `compare` → `noise-fit`

12.1 Calibration: one p_{opt} per (machine, N), shared by every problem class

The embedding, i.e. the chains, depends only on the topology and on N. So the chain-strength prefactor is measured once per (machine, N), on one problem class (the pure SK one by default), and every class uses it. For each pair the grid $0.75, 1, 1.414, 2, 3, 4, 6$ is climbed from the smallest p, with 200 reads on instance 0. It stops at the first p with no broken chain in any read: that p is p_{opt}.

`dwave_utils/pipeline.py`, lines 152–170: one calibration per (machine, N)

```
152 def calibrate(out: Path, cfg, budget_s: float | None = None, check: bool = False, threshold: float = 0.0,
153                key=None, log=print) -> dict:
154     """ONE p_opt per (machine, N), shared by every problem class: the embedding depends only on the
155     topology and N. Measured on one class (default the pure SK one): adaptive climb of GRID, 200 reads,
156     instance 0. Stored in config.json as chain_strength_by_N; any per-class table is removed."""
157     import main_simulation as ms
158     from dwave_utils import coupling_scan as cs
159     key = calibration_class(cfg, key)
160     log(f"[calibrate] shared calibration per (machine, N), measured on problem class {key}")
161     cs.find_best_coupling_per_problem(out, cfg, cfg.Ns, prefactors=GRID, num_reads=CAL_READS, keys=[key],
162                                       threshold=threshold, budget=ms.QPUBudget(budget_s),
163                                       check_only=check, apply=True, adaptive=True, log=log)
164     if check:
165         return {}
166     best = (ms.load_config(out).chain_strength_by_class or {}).get(key) or {}
167     ms.update_config(out, cfg, chain_strength_by_N=best, chain_strength_by_class={}, calibration_class=key)
168     if best:
169         _write_table(out, {key: best}, log)
170     return best
```

`run` refuses to start while some (machine, N) has no p_{opt}:

`dwave_utils/pipeline.py`, lines 221–225: what is still uncalibrated

```
221 def missing_calibration(out: Path, cfg, backends=None) -> list:
222     import main_simulation as ms
223     bes = backends or _qpu_backends(out, cfg)
224     t = cfg.chain_strength_by_N or {}
225     return [(be.label, N) for be, N in _pairs(cfg, bes) if str(N) not in (t.get(be.label) or {})]
```

`dwave_utils/coupling_scan.py`, lines 63–69: the p used by every QPU call

```
63 def prefactor_for(cfg, label: str, key: str, N: int) -> float:
64     """p for (back-end, problem class, N): the per-class calibration if there is one for this class and
65     back-end, else the per-back-end table (chain_strength_by_N), else the campaign default."""
66     by_class = (getattr(cfg, "chain_strength_by_class", None) or {}).get(key) or {}
67     if by_class.get(label):
68         return coupling_for(by_class, label, N, cfg.chain_strength_prefactor)
69     return coupling_for(cfg.chain_strength_by_N or {}, label, N, cfg.chain_strength_prefactor)
```

The cost is about 33 (machine, N) pairs $\times$ 5 values of p $\times$ 55.8 ms $\approx$ 9 s.

12.2 Spending the whole budget: `budget --fill` and `run --total`

`--total` is the QPU time available when the campaign started. What the campaign has already spent is read from its own results: the main runs, the calibration and the shimming. The cost of a call is linear in the reads, $t(R) = a + bR$, with a and b taken from the solver's timing model. `--fill` picks the largest R that fits:

$$R = \left\lfloor \frac{T_{\text{total}} - T_{\text{spent}} - T_{\text{calibration left}} - T_{\text{margin}} - \sum_{\text{calls}} a}{\sum_{\text{calls}} b} \right\rfloor,$$

capped so that a single call stays below the QPU's 1 s limit. `run --total` caps the run at $T_{\text{total}} - T_{\text{spent}}$. It goes N by N, both machines at each N, so a stop leaves complete sizes behind.

12.3 Report and compare

`report` writes one row per (machine, class, N, instance): the p used, the best configuration and its energy and reward, and the reference (exact for $N \leq 24$). `compare` puts the QPU, the classical solvers and the game side by side, with classes from easy to hard. Its metrics are:

- the residual $\rho = (R_{\text{ref}} - R)/|R_{\text{ref}}|$ of the best sample and of the typical sample;
- the exact success $P(\rho = 0)$, meaningful only where the optimum is certified;
- the *satisficing* success $P(\rho \leq \delta)$, the main metric at large N;
- the time to target $\text{TTT}_{99}(\delta) = t \ln 0.01 / \ln(1 - P)$.

Above $N = 24$ the reference is the best known one, and its plausibility is checked against the Parisi curve $R(N) \approx -2(-0.7633 + 0.7N^{-2/3})$.

12.4 Feasibility: can the QPU represent the problem at all?

All quantities are in field units $u = J_{\text{rms}}\sqrt{N-1}$.

- **Chains must hold:** $p_{\text{low}} = \max_i |h_i(S^*)|/u$, the largest field at a low-energy state.
- **Divisor:** $D = \max(c/r, \max|W|)$, with $c = p_{\text{low}}u$ and $r = 2$ with the extended range, otherwise 1.
- **Relative control error:** $\varepsilon = \sigma\sqrt{|E|}\, D/J_{\text{rms}}$.
- **Temperature over the critical one:** $T_u/T_c = D/(\beta_{\text{eff}} u T_c)$.

The verdict is YES if $\varepsilon \leq 0.3$ and $T_u/T_c \leq 0.3$, and NO if either is ≥ 1 or if N cannot be embedded.

`dwave_utils/feasibility.py`, lines 83–125: the verdict

```python
def assess(topology: str, N: int, W_list, sigma=0.02, beta=5.0, extended=False, eps_ok=0.3, t_ok=0.3,
           T_c=1.0, rng=None) -> dict:
    rng = rng or np.random.default_rng(0)
    emb = embedding_stats(topology, N)
    row = {"topology": topology, "N": N, "largest_N": emb["largest_N"], "sigma": sigma, "beta_eff": beta,
           "extended_range": extended}
    if not emb["fits"]:
        row.update(verdict="NO", reason=f"N > largest clique {emb['largest_N']} of {topology}")
        return row
    r = 2.0 if extended else 1.0
    fb = [field_bound(W, rng) for W in W_list]
    p = float(np.mean([f["p_low"] for f in fb]))
    u = float(np.mean([f["u"] for f in fb]))
    jr = float(np.mean([f["J_rms"] for f in fb]))
    wmax = float(np.mean([f["max_abs_W"] for f in fb]))
    c = p * u
    D = max(c / r, wmax / 1.0)
    prog = jr / D                                    # rms programmed logical coupling
    eps = sigma * np.sqrt(emb["E_mean"]) / prog
    T_u = D / (beta * u)
    row.update(chain_len_mean=emb["chain_len_mean"], chain_len_max=emb["chain_len_max"], E_mean=emb["E_mean"],
               p_low=p, p_typ=float(np.mean([f["p_typ"] for f in fb])), c_over_Jrms=c / jr,
               c_over_maxW=c / wmax, divisor=D, programmed_rms=prog, eps=float(eps), T_u=float(T_u), T_c=T_c,
               T_ratio=float(T_u / T_c),
               sigma_needed=float(eps_ok * prog / np.sqrt(emb["E_mean"])),
               beta_needed=float(D / (u * t_ok * T_c)))
    if eps >= 1 or T_u / T_c >= 1:
        why = []
        if eps >= 1:
            why.append(f"control error {eps:.2f} x the couplings")
        if T_u / T_c >= 1:
            why.append(f"temperature {T_u / T_c:.2f} x T_c")
        row.update(verdict="NO", reason="; ".join(why))
    elif eps <= eps_ok and T_u / T_c <= t_ok:
        row.update(verdict="YES", reason="faithful and cold")
    else:
        why = []
        if eps > eps_ok:
            why.append(f"control error {eps:.2f} > {eps_ok}")
        if T_u / T_c > t_ok:
            why.append(f"T/T_c {T_u / T_c:.2f} > {t_ok}")
        row.update(verdict="MARGINAL", reason="; ".join(why))
    return row
```

12.5 Measuring σ and β_{eff}: `noise-fit`

The model of one call: the chip holds $W/D + \delta$, with $\delta_{ij} \sim \mathcal{N}(0, \sigma^2|E_{ij}|)$ fixed for the whole call, and every read is Boltzmann distributed at β_{eff}:

$$P(S \mid \delta) = \frac{e^{-\beta_{\text{eff}} E_\delta(S)}}{Z_\delta}, \qquad L(\beta, \sigma) = \mathbb{E}_\delta\Big[\prod_{\text{reads}} P(S_r \mid \delta)\Big].$$

For $N \le 12$ all 2^{N-1} states are enumerated, so Z is exact. The integral over δ uses the Laplace approximation around the most likely δ, found by Newton's method. The log-likelihoods of all calls of a machine are summed on a grid, which is then refined; the 95 % interval is $\Delta \log L = 1.92$.

`dwave_utils/noise_fit.py`, lines 58–117: likelihood of one call

```
def call_loglik(W: np.ndarray, D: float, Ecount: np.ndarray, S_obs: np.ndarray, counts: np.ndarray,
                betas=BETAS, sigmas=SIGMAS, **_ignored) -> np.ndarray:
    """log L(beta, sigma) of one call's reads, shape (len(betas), len(sigmas)), integrating the unknown
    coupler errors delta with a Laplace approximation (accurate when the call has many reads):

      g(delta) = beta o.(w + delta) - R log Z(beta (w + delta))      o = sum_r B(S_r), w = (W/D)_pairs
      phi(delta) = g(delta) - sum delta^2 / (2 sigma^2 e)             e = |E_ij|
      log L ~ phi(delta*) - 1/2 log det(I + sigma^2 diag(e) beta^2 R Cov_delta*(B))
    with B(S) = (S_i S_j)_{i<j} and delta* found by Newton's method (warm-started along sigma)."""
    N = len(W)
    states = all_states(N)
    B, iu = _bonds(states)
    S_obs = np.asarray(S_obs, float) * np.sign(np.asarray(S_obs, float)[:, :1])
    idx = ((1.0 - S_obs[:, 1:]) / 2.0).astype(np.int64) @ (1 << np.arange(N - 1))
    counts = np.asarray(counts, float)
    R = counts.sum()
    o = counts @ B[idx]
    w = (W / D)[iu]
    e = np.maximum(Ecount[iu], 1.0)
    out = np.empty((len(betas), len(sigmas)))

    def logZ_mu_cov(theta, need_cov):
        a = B @ theta
        m = a.max()
        p = np.exp(a - m)
        Z = p.sum()
        p /= Z
        mu = p @ B
        cov = (B * p[:, None]).T @ B - np.outer(mu, mu) if need_cov else None
        return m + np.log(Z), mu, cov

    for jb, b in enumerate(betas):
        delta = np.zeros_like(w)
        for js, sg in enumerate(sigmas):
            if sg == 0:
                lz, _, _ = logZ_mu_cov(b * w, False)
                out[jb, js] = b * o @ w - R * lz
                continue
            prec = 1.0 / (sg * sg * e)
            for _ in range(30):                                          # Newton on phi (concave)
                lz, mu, cov = logZ_mu_cov(b * (w + delta), True)
                grad = b * (o - R * mu) - prec * delta
                H = b * b * R * cov + np.diag(prec)                      # -Hessian, positive definite
                step = np.linalg.solve(H, grad)
                phi0 = b * o @ (w + delta) - R * lz - 0.5 * prec @ delta ** 2
                t = 1.0
                while t > 1e-4:                                          # backtracking line search
                    nd = delta + t * step
                    lz1, _, _ = logZ_mu_cov(b * (w + nd), False)
                    if b * o @ (w + nd) - R * lz1 - 0.5 * prec @ nd ** 2 >= phi0 - 1e-12:
                        break
                    t /= 2
                delta = delta + t * step
                if np.max(np.abs(t * step)) < 1e-9:
                    break
            lz, mu, cov = logZ_mu_cov(b * (w + delta), True)
            phi = b * o @ (w + delta) - R * lz - 0.5 * prec @ delta ** 2
            M = np.eye(len(w)) + (sg * sg * e)[:, None] * (b * b * R * cov)
            out[jb, js] = phi - 0.5 * np.linalg.slogdet(M)[1]
```

```
    return out
```

On synthetic data with known values, 30 calls of 1000 reads, the fit recovers them: $(\beta, \sigma) = (8, 0.03) \to (7.8, 0.031)$; $(3, 0.01) \to (2.96, 0.011)$; $(15, 0) \to (15.1, 0.0004)$. `feasibility --from-fit DIR` then uses the measured values instead of the assumed ones.

```
python -m dwave_utils noise-fit   --out results\sk                    # sigma, beta_eff
    per machine
python -m dwave_utils feasibility --topology zephyr --N 16 64 104 --from-fit results\sk
```

13 Reproducibility: everything needed to reconstruct a call

Each call saves:

- the problem id and label (findable in the Leap dashboard);
- the solver identity, ranges and a fingerprint of the working graph at that moment;
- the embedding actually used;
- the chain strength;
- the exact physical problem (`N...._physical.npz`: qubits, couplers, values before scaling) and the auto-scale divisor;
- every distinct configuration read, with its reward, polished reward and number of broken chains.

The instances (full J matrices) and the configuration are saved once per campaign. `RESULTS_FORMAT.md` documents every field.

`python main_simulation.py validate --out DIR` re-derives everything from these inputs:

- the reward of every stored configuration, and the exact optimum for $N \leq 24$;
- the read counts, and the broken-chain counts against D-Wave's `chain_break_fraction`;
- the chain strength from the formula;
- the physical problem: re-embedding the instance with the saved embedding and chain strength reproduces the saved one exactly;
- that every embedding is a valid minor of K_N on the solver's own topology.

`main_simulation.py`, lines 931–1038: `validate`

```python
def validate(out: Path, cfg: CampaignConfig, log=print) -> dict:
    """Re-derive every stored number from the stored inputs, a posteriori.

    For each annealer result: rebuild the instance from instances/*.npz and recompute the reward of
    every stored configuration (reads.S, best_S, polished_best_S); check read counts, the broken-
    chain histogram against D-Wave's chain_break_fraction, the chain strength against
    p * J_rms * sqrt(N-1), the embedding_used (valid minor of K_N on the ideal graph of the
    topology, which contains every working graph) and its chain lengths. For each classical
    file: recompute the reward of the reference configuration and, for N <= n_exact, re-run the
    exact enumeration. Returns {check: [passed, failed]}."""
    from skgame.optim.embedding import is_valid_embedding, topology as topo
    from skgame.optim.backends import parse_spec
    tally = {}

    def check(name, ok, detail=""):
        t = tally.setdefault(name, [0, 0])
        t[0 if ok else 1] += 1
        if not ok:
            log(f"[validate] FAILED {name}: {detail}")

    def close(a, b):
```

```
        return abs(a - b) <= 1e-9 * max(1.0, abs(b))

    graphs = {}
    for key, eps, J0 in _classes(cfg):
        for N in cfg.Ns:
            if not instance_path(out, key, N).exists():
                continue
            J = ensure_instances(out, cfg, key, eps, J0, N)
            cp = classical_path(out, key, N)
            if cp.exists():
                c = json.loads(cp.read_text())
                for b in range(cfg.n_instances):
                    P = PlannerProblem(J[b])
                    if c.get("ref_S"):
                        S = np.array([1.0 if ch == "+" else -1.0 for ch in c["ref_S"][b]])
                        check("classical reference configuration -> reference value", close(P.reward(S), c["ref"][b]),
                              f"{key} N={N} i={b}")
                    if c["ref_exact"] and N <= 24:
                        check("exact optimum re-computed", close(ExactEnumeration(max_N=24).solve(P).reward, c["ref"][b]),
                              f"{key} N={N} i={b}")
            for lab in sorted(p.name for p in (out / "anneal").glob("*")) if (out / "anneal").exists() else []:
                for b in range(cfg.n_instances):
                    ap = anneal_path(out, lab, key, N, b)
                    if not ap.exists():
                        continue
                    r = json.loads(ap.read_text())
                    info, P, where = r["info"], PlannerProblem(J[b]), f"{lab} {key} N={N} i={b}"
                    to_S = lambda st: np.array([1.0 if ch == "+" else -1.0 for ch in st])
                    if r.get("best_S"):
                        check("best configuration -> best reward", close(P.reward(to_S(r["best_S"])), r["best_reward"]), where)
                    if r.get("polished_best_S"):
                        check("polished configuration -> polished reward",
                              close(P.reward(to_S(r["polished_best_S"])), r["polished_best_reward"]), where)
                    t = r.get("reads")
                    if t and t.get("S"):
                        rec = np.atleast_1d(P.reward(np.array([to_S(x) for x in t["S"]]))) if t["S"] else np.zeros(0)
                        check("every stored configuration -> its reward", np.allclose(rec, t["reward"], rtol=1e-9, atol=1e-12), where)
                    if t:
                        check("read counts add up", sum(t["count"]) == info["num_reads"] - info.get("discarded_reads", 0)
                              or sum(t["count"]) == info["num_reads"], where)
                    h = info.get("broken_chains_hist")
                    if h and "chain_break_fraction_mean" in info and not info.get("discarded_reads"):
                        mean = sum(int(k) * v for k, v in h.items()) / max(1, sum(h.values())) / N
                        check("broken-chain counts agree with D-Wave's chain_break_fraction",
                              abs(mean - info["chain_break_fraction_mean"]) < 1e-9, where)
                    if info.get("chain_strength") is not None:
                        q = P.W[np.triu_indices(N, 1)]
                        expect = info["settings"]["chain_strength_prefactor"] * np.sqrt(np.mean(q ** 2)) * np.sqrt(N - 1)
                        check("chain strength = p J_rms sqrt(N-1)", close(info["chain_strength"], expect), where)
                    pf = ap.with_name(info["physical_problem_file"]) if info.get("physical_problem_file") else None
                    if pf is not None and pf.exists() and info.get("embedding_used"):
                        import dimod
                        from dwave.embedding import EmbeddedStructure
                        ph = np.load(pf)
                        emb = {int(k): v for k, v in info["embedding_used"].items()}
                        edges = list(zip(ph["u"].tolist(), ph["v"].tolist()))
                        rebuilt = EmbeddedStructure(edges, emb).embed_bqm(P.to_bqm(), chain_strength=info["chain_strength"],
                                                                          smear_vartype=dimod.SPIN)
                        saved = dimod.BinaryQuadraticModel(dict(zip(ph["h_q"].tolist(), ph["h"].tolist())),
                                                           dict(zip(edges, ph["J"].tolist())), float(ph["offset"]), "SPIN")
                        check("physical problem = embed(instance, embedding_used, chain_strength)",
                              rebuilt.is_almost_equal(saved), where)
                        sc = info["programmed"].get("scaling", {"mode": "auto_scale"})
                        if sc["mode"] == "extended_j_range":
                            own = {q: i for i, ch in emb.items() for q in ch}
```

```
                            chain = np.array([abs(v) for (x, y), v in zip(edges, ph["J"]) if own[x] == own[y]])
                            logi = np.array([abs(v) for (x, y), v in zip(edges, ph["J"]) if own[x] != own[y]])
                            expect = max(chain.max() / -sc["ext_lo"], logi.max() if logi.size else 0.0)
                            check("extended-range divisor = max(c/|ext_lo|, max logical |J|)",
                                  close(sc["divisor"], expect), where)
                        else:
                            check("auto_scale divisor = max |J| of the physical problem",
                                  close(float(np.abs(ph["J"]).max()), info["programmed"]["
    max_abs_J_before_scaling"]), where)
                    if info.get("embedding_used"):
                        emb = {int(k): v for k, v in info["embedding_used"].items()}
                        t = (info.get("solver_properties") or {}).get("topology") or {}
                        tp = t.get("type") or parse_spec(info["backend"])[1]
                        shape = tuple(t["shape"]) if t.get("shape") else topo(tp).shape
                        from skgame.optim.embedding import Topology
                        G = graphs.setdefault((tp, shape), Topology(tp, shape, "").ideal_graph())
                        check("embedding is a valid minor of K_N", is_valid_embedding(emb, N, G), where)
                        check("chain lengths match the embedding", [len(emb[i]) for i in range(N)] == info["
    chain_lengths"], where)
    for name, (ok, bad) in tally.items():
        log(f"[validate] {'OK  ' if not bad else 'FAIL'} {name}: {ok} passed, {bad} failed")
    if not tally:
        log("[validate] nothing to validate yet")
    return tally
```

14 Ising or QUBO? And is the problem trivial?

Ising, which is equivalent to QUBO. The QPU's native form is Ising: spins $S_i = \pm 1$, fields h_i, couplings J_{ij}. The project sends the SK problem in this form, with $h = 0$ and $J_{ij} = -W_{ij}$ on all $N(N-1)/2$ pairs (`to_bqm`). A QUBO is the same model in binary variables $x_i \in \{0, 1\}$. With $S_i = 2x_i - 1$:

$$\begin{aligned} H &= -\sum_{i<j} W_{ij} S_i S_j = -\sum_{i<j} W_{ij}(2x_i - 1)(2x_j - 1) \\ &= \sum_{i<j} (-4W_{ij})\, x_i x_j + \sum_i \Big(2 \sum_{j \neq i} W_{ij}\Big) x_i - \sum_{i<j} W_{ij}. \end{aligned} \tag{5}$$

So $Q_{ij} = -4W_{ij}$, $Q_{ii} = 2\sum_{j \neq i} W_{ij}$, and the offset is $-\sum_{i<j} W_{ij}$. The minimiser is the same (via $x = (S+1)/2$), and so is every energy. `PlannerProblem.to_qubo` returns this form. A test checks it against the Ising form on all 2^9 configurations of an instance, and checks that dimod's `change_vartype` turns one into the other. Sending the QUBO instead would change nothing: dimod converts it to the same Ising problem.

`skgame/optim/problem.py`, lines 52–64: the QUBO form

```
    def to_qubo(self):
        """The same problem as a QUBO in binary variables x_i in {0, 1}, with S_i = 2 x_i - 1.

        H(S) = -sum_{i<j} W_ij S_i S_j
             = -sum_{i<j} W_ij (2x_i - 1)(2x_j - 1)
             = sum_{i<j} (-4 W_ij) x_i x_j + sum_i (2 sum_{j != i} W_ij) x_i - sum_{i<j} W_ij,
        so Q_ij = -4 W_ij (i < j), Q_ii = 2 sum_{j != i} W_ij, offset = -sum_{i<j} W_ij.
        Same minimiser (mapped by x = (S + 1) / 2) and same energy. The QPU works in Ising form;
        dimod converts between the two exactly (bqm.change_vartype)."""
        iu = np.triu_indices(self.N, 1)
        Q = {(int(i), int(j)): -4.0 * float(self.W[i, j]) for i, j in zip(*iu)}
        Q.update({(i, i): 2.0 * float(self.W[i].sum()) for i in range(self.N)})
        return Q, -float(self.W[iu].sum())
```

Is it trivial? The problem is not:

- the SK ground state is a dense spin glass with frustration and exponentially many local minima. Its worst case is NP-hard, and typical instances need exponential time for every known exact method;

- there is no field ($h = 0$), so S and $-S$ are both ground states. The code counts either as success.

What *is* trivial is the 1-second test: $N = 8, 16$ is solved exactly by a laptop in milliseconds. It tests the machinery (token, embedding, timing, budget), not the QPU's ability. The campaign's real question starts at large N, up to 104 on Zephyr and 177 on Pegasus. There, good classical solvers (parallel tempering, tabu) remain very strong on dense SK, and the QPU is handicapped by the all-to-all embedding: long chains, couplings compressed to ~ 0.05, analogue noise. The honest expectation is not that the QPU wins. The campaign measures where it stops finding the optimum, how that depends on broken chains, and whether Zephyr's shorter chains move that point. The `ferro_easy` regime ($J_0 = 3$) is *deliberately* easy: it is the control case where every method should succeed.

15 From reads to results

`main_simulation.py`, lines 532–563: per back-end metrics in `collect`

```
532             for s, recs in ann.items():
533                 done = [(b, r) for b, r in enumerate(recs) if r is not None]
534                 if not done:
535                     continue
536                 p = np.array([np.sum(np.array(r["counts"])[_success(r["rewards"], ref[b])]) / max(1, np.sum(r["
    counts"]))
537                               for b, r in done])
538                 best_res = np.array([(ref[b] - r["best_reward"]) / abs(ref[b]) for b, r in done])
539                 mean_res = np.array([np.average((ref[b] - np.array(r["rewards"])) / abs(ref[b]), weights=r["
    counts"])
540                                      if np.sum(r["counts"]) else np.nan for b, r in done])
541                 t_read = np.median([_time_per_read(r) for b, r in done])
542                 acc = [r["info"].get("timing_us", {}).get("qpu_access_time", np.nan) / 1e6 for b, r in done]
543                 cbf = [r["info"].get("chain_break_fraction_mean", np.nan) for b, r in done]
544                 extra = {}
545                 if all("polished_rewards" in r for b, r in done):
546                     pp = np.array([np.sum(np.array(r["polished_counts"])[_success(r["polished_rewards"], ref[b])
    ])
547                                    / max(1, np.sum(r["polished_counts"])) for b, r in done])
548                     extra.update(p_success_polished=float(np.median(pp)),
549                                  residual_polished=float(np.median([(ref[b] - r["polished_best_reward"]) / abs(
    ref[b])
550                                                                     for b, r in done])))
551                 emb = [r["info"].get("embedding") for b, r in done]
552                 if all(emb):
553                     extra.update(chain_len_mean=float(np.mean([e["chain_len_mean"] for e in emb])),
554                                  chain_len_max=int(max(e["chain_len_max"] for e in emb)),
555                                  n_qubits=int(np.mean([e["n_qubits"] for e in emb])),
556                                  chain_strength=_nan_to_none(float(np.mean([r["info"].get("chain_strength") or
    np.nan
557                                                                             for b, r in done]))))
558                 rows.append(dict(base, kind="anneal", method=f"anneal_{s}", regime="", n_done=len(done), **extra
    ,
559                                  p_success_median=float(np.median(p)), p_success_mean=float(p.mean()),
560                                  p_best_found=float(np.mean(best_res <= TOL)),
561                                  residual_median=float(np.median(best_res)), residual_mean_read=float(np.
    nanmedian(mean_res)) if np.isfinite(mean_res).any() else None,
562                                  seconds_median=t_read, tts99=_tts(p.mean(), t_read),
563                                  qpu_access_s=float(np.nansum(acc)), chain_break_fraction=_nan_to_none(float(np.
    mean(cbf)))))
```

For each problem class, N and instance b, with reference R_{ref} (exact for $N \leq 24$; otherwise the best

of the classical solvers and of every annealer read, polished reads included):

$$p_{\text{success}} = \frac{\#\{\text{reads with } R \geq R_{\text{ref}} - 10^{-9}\}}{n_{\text{reads}}} \quad \text{(per read)}, \tag{6}$$

$$\text{residual} = \frac{R_{\text{ref}} - R_{\text{best read}}}{|R_{\text{ref}}|}, \tag{7}$$

$$t_{\text{read}} = \frac{t_{\text{sampling}}}{n_{\text{reads}}} \quad (t_{\text{sampling}} = \texttt{qpu_sampling_time}), \tag{8}$$

$$\text{TTS}_{99} = t_{\text{read}} \frac{\ln 0.01}{\ln(1-p)}. \tag{9}$$

`main_simulation.py`, lines 582–597: time per read and TTS_{99}

```
582 def _time_per_read(r) -> float:
583     """QPU: sampling time per read (anneal + readout + delay); simulator: wall time per read."""
584     t = r["info"].get("timing_us", {})
585     n = r["info"]["num_reads"]
586     if "qpu_sampling_time" in t:
587         return t["qpu_sampling_time"] / 1e6 / n
588     return r["info"]["wall_seconds"] / n
589
590
591 def _tts(p: float, t: float) -> float:
592     """Time to reach the reference with 99 % confidence: t ln(0.01) / ln(1 - p)."""
593     if p >= 1:
594         return float(t)
595     if p <= 0:
596         return INF
597     return float(t * math.log(0.01) / math.log1p(-p))
```

`compare_backends` ranks the back-ends of the same device for each (class, N): higher median p_{success} wins, ties go to the lower residual. The output is `summary/backend_comparison.csv` and `.png`.

16 How the D-Wave path was tested

- `test_embedding.py`: clique embeddings are valid on both topologies (every chain connected, chains disjoint, every pair linked); Zephyr chains are shorter at production size; the cache round-trips; back-end specs parse; the embedded simulator finds the exact ground state, and the chain strength equals $p\,J_{\text{rms}}\sqrt{N-1}$ computed independently.
- `test_campaign.py`: every read is kept and `seed` is skipped where unsupported (with `ExactSolver`); the whole campaign runs end to end and resumes; **both QPU back-ends run against** `MockDWaveSampler` with Zephyr and Pegasus topologies, using the real `DWaveSampler` code path from `solver={topology__type}` onwards; the budget guard (rule, refusal, completion, `--spent`) is checked.
- What the mock cannot test: the real timing values, the real noise, SAPI errors and Leap accounting. That is what the one-second test below is for.

17 The one-second test, and a plan for the 180 s

17.1 Test: at most 1 s of QPU time, both topologies

A tiny separate campaign: $N = 8$ and 16, one instance, one problem class, 100 reads per call, both QPUs. That makes 4 calls. At 100 reads each the timing model gives a few tens of ms per call, so the whole test costs a fraction of a second. The guard makes **1 s a hard ceiling** anyway.

```
cd ...\Dimod\src\unlearnable_games_explanation      # (PowerShell, .venv active)
$T = "results\qpu_test"
```

```
python main_simulation.py init --preset smoke --out $T --Ns 8,16 --n-instances 1 --regimes
    sg_satisficing --num-reads 100
python main_simulation.py qpu --out $T --check                  # predicted cost: spends
    nothing
python main_simulation.py qpu --out $T --budget-seconds 1       # the test: <= 1 s,
    normally ~0.1-0.2 s
python main_simulation.py qpu --out $T --spent                  # what Leap charged, and
    actual/estimate
python main_simulation.py classical --out $T                    # seconds, for the
    reference optimum
python main_simulation.py analyze --out $T                      # success per read, chain
    breaks
```

What to check in the output:

1. every call prints `qpu_access=...ms (est ...ms)`: real vs predicted;
2. `--spent`: the total, well below 1 s, and `actual/estimate` close to 1;
3. `chain_breaks`: should be 0 or tiny at $N = 8, 16$;
4. the Leap dashboard shows the same usage.

17.2 Spending the 180 s before 4 November

The plan uses the pipeline of Sec. 12. Calibration takes about 9 s, and `--fill` gives everything that is left to the main run:

```
$F = "results\sk"
python main_simulation.py init --preset full --out $F --Ns 8:160:8 --n-instances 5
python -m dwave_utils budget    --out $F --total 178
python -m dwave_utils calibrate --out $F --budget-seconds 15
python -m dwave_utils budget    --out $F --total 178 --fill
python -m dwave_utils run       --out $F --total 178
python -m dwave_utils report    --out $F
python -m dwave_utils noise-fit --out $F                                  # no QPU
```

The classical stage and `compare` need no QPU time and can run afterwards, on the laptop or on Leonardo.

References

- D-Wave, Ocean documentation: https://docs.dwavequantum.com/en/latest/ocean/; Leap authorisation: https://docs.dwavequantum.com/en/latest/ocean/leap_authorization.html.
- D-Wave, QPU timing and `estimate_qpu_access_time`: `dwave.cloud.solver.StructuredSolver` (source read for §6, dwave-cloud-client 0.14.8).
- Boothby, King, Raymond, *Fast clique minor generation in Chimera qubit connectivity graphs*, Quantum Inf. Process. 15, 495 (2016): the clique embedding.
- Cai, Macready, Roy, *A practical heuristic for finding graph minors*, arXiv:1406.2741 (2014): minorminer.
- Cineca, Quantum Computing as a Service: https://www.quantumcomputinglab.cineca.it/en/resources.
- Project: `Dimod/explanation_project.pdf` (SK game, algorithms, embedding, chains, noise), `Dimod/LEAP_TOKEN.md`, `README_LOCAL.md`, `README_CINECA.md`.